ACME PLATFORM · HIRING ASSESSMENT

DevOps / Platform Engineer

Architecture design + hands-on exercise

Format	**Your own GitHub repo + architecture document**
Classification	**Confidential — hiring use only**

Introduction

Thank you for your interest in the Platform Engineering role at Acme Platform. This assessment evaluates how you design secure, operable infrastructure for a multi-tenant B2B SaaS product — and how you translate that design into concrete hardening work.

> **Anonymized scenario.** Do not reference real company names, products, customer identities, or production domains. Use placeholders such as *Acme Platform*, *cell-01*, and `platform.example.com`.

You will receive a link to a starter repository. **Copy it to your own GitHub repository** (fork, duplicate, or push to a new private repo) and complete all work there. When finished, send reviewers the URL to your repository and grant them read access.

Do not open pull requests or push branches to the starter repository provided by the hiring team.

Assessment structure

Part	Focus	Weight
A	Architecture & security design	60%
B	Hands-on hardening exercise	40%

Part A is a written design exercise. Part B asks you to fix deliberately insecure Terraform, Helm, and CI/CD scaffolding in the starter repository.

PART A · 60%

Architecture & Security Design

Scenario

Acme Platform is a B2B SaaS product used by enterprise customers who process high-value intellectual property and regulated data. Propose a **production-ready platform architecture** that supports three commercial tiers while meeting **SOC 2 Type II** and **GDPR** requirements.

Application stack

- ~10 microservices (API gateway + domain services), containerized on Node.js/NestJS
- Data stores: PostgreSQL, Redis, S3-compatible object storage
- Workload patterns: async AI jobs, export pipelines, WebSocket realtime
- Tooling: GitHub Actions, Docker, Helm · Primary region: **AWS eu-central-1**

Commercial tiers

Tier	Deployment model	Typical customers
Standard	Shared multi-tenant SaaS	SMB
Enterprise	Dedicated stack per customer	Large enterprise
Regulated	Dedicated stack + data residency controls	Finance, luxury, EU-only

Assume the platform will eventually operate **20+ dedicated customer cells** alongside the shared tier. Your design must scale operationally — not just technically.

Design questions

Answer each section in your architecture document. Be specific: name AWS services, isolation boundaries, and operational trade-offs. Include diagrams where noted.

A1 — Deployment models by tier

For Standard, Enterprise, and Regulated tiers, describe what is shared vs. isolated, ingress model, backup strategy with target RTO/RPO, and relative cost/complexity. Present as a comparison table plus short narrative per tier.

A2 — Network & access model *(critical)*

Actor	Requirement
End users	Public access via API gateway / web app only
Platform engineers	No public Kubernetes API; VPN or Zero Trust access

Actor	Requirement
Customer support	Access to one dedicated instance without cross-tenant exposure
Service-to-service	Private networking with segmentation

Define an **egress control policy** for external dependencies (payments, model APIs, email) and how it is enforced.

Deliverable: network diagram — VPC, subnets, NAT, VPC endpoints, VPN/Zero Trust, bastion/SSM.

A3 — Identity, secrets & encryption keys

- IAM least privilege for humans, CI/CD, and workloads
- Secrets storage, injection, and rotation
- CI/CD auth via OIDC (not long-lived keys)
- BYOK/CMEK for Regulated tier
- Break-glass access: approval, scope, audit trail

A4 — Kubernetes & runtime platform

- Shared vs. dedicated cluster/account per tier
- Namespace isolation, network policies, service accounts
- Platform components (ingress, cert-manager, external-secrets, policy engine)
- Baseline hardening: pod security, admission control, image scanning

A5 — Data layer

PostgreSQL, Redis, and object storage — placement per tier, encryption, migration strategy for dedicated cells, backup/restore, cross-region for Regulated tier.

A6 — CI/CD & release strategy

Release flow across shared SaaS and 20+ customer cells: promotion, canary/progressive rollout, rollback, and a disaster-recovery drill procedure.

A7 — Observability & SOC 2 evidence

Logs, metrics, traces; alerting approach; retention and access controls; audit evidence artifacts mapped to CC6, CC7, CC8, A1.

A8 — Enterprise customer onboarding

Runbook from contract signed through offboarding: provisioning, DNS/TLS, deployment, smoke tests, support handoff, data deletion.

Part A deliverables

Submit as `docs/architecture-proposal.md` (preferred) or PDF. Target: **5–10 pages**.

1. Executive summary

2. Architecture diagram
3. Network diagram
4. Tier comparison table
5. SOC 2 control mapping (CC6, CC7, CC8, A1)
6. Top 5 risks & trade-offs
7. Implementation roadmap

PART B · 40%

Hands-on Technical Exercise

Harden the intentionally insecure starter repository for Enterprise customer cell `cell-01`, focusing on the `billing-service` workload.

B1 — Terraform

- Remove AdministratorAccess
- RDS `storage_encrypted = true`
- No public K8s API / SSH
- Remote state S3 + DynamoDB
- S3 encryption + public access block
- Outputs: RDS, S3, IAM ARNs

B2 — Helm chart

- Production env vars & SSL mode
- Secrets via secretKeyRef
- securityContext + probes
- Resource limits
- NetworkPolicy (api-gateway only)
- `values.production.yaml`

B3 — CI/CD

- `deploy.yml` on main
- OIDC auth, ECR push
- Trivy CRITICAL fail
- npm audit
- Helm deploy to cell-01
- Manual approval gate

B4 + B5 — Script & security doc

- `provision-cell.sh` with audit logging
- Params: CUSTOMER_ID, AWS_REGION, TIER
- `SECURITY.md`: 3 SOC 2 controls, 5 risks, rotation runbook

Submission checklist

- ☐ Starter repository copied to your own GitHub repository
- ☐ Part A document in `docs/`
- ☐ Part B code changes committed
- ☐ README updated with known limitations and setup notes
- ☐ Repository URL shared with reviewers (read access granted)

Confidential — for hiring use only. Do not distribute.